2. *Authentication* is about verifying the user's claimed identity through factors like a passport, a driver's license, a password, a certificate, or a token. When multiple factors are used to verify the user's identity, we talk about *multi-factor authentication*.
3. *Authorization* always happens after authentication, and it checks what the user is allowed to do in a given context.

This chapter and the next one will cover implementing access control systems in cloud native applications. You'll see how to add authentication to a system like Polar Bookshop and use a dedicated identity and access management solution like Keycloak. I'll show you how to use Spring Security to secure applications and adopt standards like JWT, OAuth2, and OpenID Connect. In the process, you'll also add an Angular frontend to the system and learn the best practices for security when a single-page application (SPA) is involved.

NOTE The source code for the examples in this chapter is available in the Chapter11/11-begin and Chapter11/11-end folders, which contain the initial and final states of the project (https://github.com/ThomasVitale/cloud-native-spring-in-action).

11.1 Understanding the Spring Security fundamentals

Spring Security (https://spring.io/projects/spring-security) is the de facto standard for securing Spring applications, supporting imperative and reactive stacks. It provides authentication and authorization features as well as protection against the most common attacks.

The framework provides its main functionality by relying on *filters*. Let's consider a possible requirement for adding authentication to a Spring Boot